



华中科技大学

操作系统原理课程实验报告

姓 名：郝依凝
学 院：计算机科学与技术学院
专 业：计算机科学与技术
班 级：CS2304
学 号：U202315653
指导教师：谢美意

分数	
教师签名	

2025 年 12 月 30 日

目 录

实验一 lab1_challenge1 打印用户程序调用栈	1
1.1 实验目的	1
1.2 实验内容	1
1.3 实验调试及心得	6
实验二 lab2_challenge1 复杂缺页异常	8
2.1 实验目的	8
2.2 实验内容	8
2.3 实验调试及心得	10
实验三 lab3_challenge1 进程等待和数据段复制	11
3.1 实验目的	11
3.2 实验内容	11
3.3 实验调试及心得	14
实验四 lab4_challenge1 相对路径	15
4.1 实验目的	15
4.2 实验内容	15
4.3 实验调试及心得	19

实验一 lab1_challenge1 打印用户程序调用栈

1.1 实验目的

本实验旨在深入理解操作系统内核中的异常处理机制和程序调试工具的实现原理，通过实现用户程序调用栈回溯功能，增强对程序执行流程、栈帧结构、ELF 文件结构的掌握。实验要求修改 PKE 内核，使其能够在用户程序触发系统调用时，动态追踪并输出函数调用的层次结构。

这个实验需要理解系统调用与异常处理的路径，掌握用户栈与内核栈的切换机制，分析函数调用的栈帧结构，解析 ELF 文件的调试信息并主要学习 ELF 文件中符号表（.symtab）和字符串表（.strtab）的结构与格式从而将回溯得到的虚拟地址映射到源程序中的函数名，实现可配置的回溯深度控制。本次实验中，我们通过综合运用系统调用、栈帧分析、ELF 文件解析等底层知识，提升在复杂系统环境下进行问题定位与调试的能力。实验不仅加深我们对操作系统内核工作机制的理解，也为后续学习性能分析、动态追踪等高级调试技术奠定基础。

1.2 实验内容

1. 实验过程

Step1.系统调用路径上的完善

根据实验参考文档以及实验代码分析可知，应用程序的实现需要依赖调用库 `user_lib.h`，即应用程序所使用的函数均在 `user_lib.c` 中具体实现，因此要实现新功能打印用户程序调用栈需要分析用户所使用函数的具体实现方式。

根据 lab1_1 分析已经可以得到实现打印用户程序调用栈具体函数调用过程：
`print_backtrace`→`do_user_call`→`ecall`（获得系统调用号并使 RISC-V 转到 S 模式的 trap 处理入口执行后陷入内核）→`smode_trap_vector`→`smode_trap_handler`→（如果是系统调用）`handle_syscall`→`do_syscall`→`sys_user_print_backtrace`

实验要求应用程序调用 `print_backtrace()` 时，应能够通过控制输入的参数（如例子 `user/app_print_backtrace.c` 中的 7）控制回溯的层数。因此在 `user_lib.h` 中声明函数 `int print_backtrace(int depth)`，再根据上述对调用过程的分析将具体实现转入 `sys_user_print_backtrace` 函数的实现。注意函数调用过程中除需要新增系统调用号 `#define SYS_user_print_backtrace (SYS_user_base + 2)` 并利用寄存器 `a0` 传递

外，需要利用寄存器 `a1` 传递参数 `depth`。

Step2.在操作系统内核中获取用户程序的栈

PKE 系统中当用户程序通过系统调用陷入到内核时，会切换到 S 模式的“用户内核”栈，而不是在用户栈上继续操作。我们的 `print_backtrace()` 函数的设计目标是回溯并打印用户进程的函数调用情况，所以，进入操作系统内核后，需要找到用户进程的用户态栈来开始回溯。`trap` 的入口处理函数首先将“进程”进行保存，即保存所有用户寄存器到 `trapframe`，因此可以得到获取用户程序栈的代码如下：

```
trapframe* tf = current->trapframe;
uint64* frame_pointer = (uint64*)tf->regs.s0;
```

Step3.根据用户态栈的结构寻址

在 64 位 RISC-V 架构中，函数调用栈帧具有标准的布局结构。当函数需要保存寄存器状态时，会在栈中分配空间。函数调用的栈帧（此时 `bar` 函数作为叶子调用函数）类似下图：

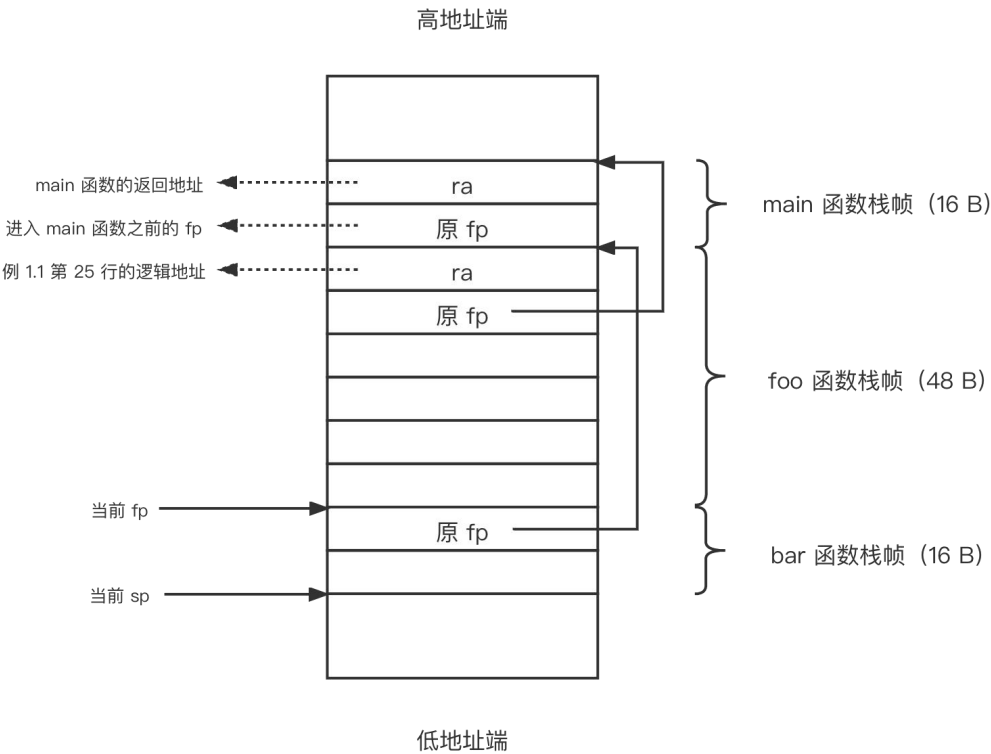


图 1 栈帧结构

由于不需要输出被调用的 `print_backtrace` 函数，因此第一次直接寻找调用 `print_backtrace` 函数的函数在用户栈中的位置，代码如下：

```
frame_pointer = (uint64*)(frame_pointer-1);
```

由于是 64 位架构，根据函数调用栈帧结构在最顶端 8B 保存函数返回地址即

可获取该函数内的一个地址，在下一个 8B 保存调用该函数的函数在函数栈帧中的位置，即可找到上一层的函数信息。而根据地址可将虚拟地址转换为源程序中的符号（具体转换方法在下面做出详细解释），具体实现代码如下：

```
for(int i = 0; i < depth; i++) {
    if(frame_pointer == 0) {
        break;
    }
    uint64 return_address = *(frame_pointer - 1);
    char* name = find_func_name(return_address);
    sprintf("%s\n", name);
    if(strcmp(name, "main") == 0) break;
    frame_pointer = (uint64*)(frame_pointer - 2);
}
```

Step4.将虚拟地址转换为源程序中的符号

为实现从地址到源程序符号的转换，需要完善框架中 ELF 文件结构，即添加 Section Header 结构 Elf64_Shdr、ELF64 符号表条目结构 Elf64_Sym。

```
typedef struct {
    uint32 sh_name; // 4 B (B for bytes)
    uint32 sh_type; // 4 B
    uint64 sh_flags; // 8 B
    uint64 sh_addr; // 8 B
    uint64 sh_offset; // 8 B
    uint64 sh_size; // 8 B
    uint32 sh_link; // 4 B
    uint32 sh_info; // 4 B
    uint64 sh_addralign; // 8 B
    uint64 sh_entsize; // 8 B
} Elf64_Shdr;
```

```
typedef struct {
    uint32 st_name;
    uint8 st_info;
    uint8 st_other;
    uint16 st_shndx;
    uint64 st_value;
    uint64 st_size;
} Elf64_Sym;
```

通过补全的结构以及在上一步获得的函数地址即可获得对应函数名，完成函数 `char* find_func_name(uint64 return_address)`。

首先获取 ELF 文件路径：

```
arg_buf arg_bug_msg;
size_t argc = parse_args(&arg_bug_msg);
const char *elf_path = arg_bug_msg.argv[0];
```

通过 spike 的 HTIF 接口打开主机上的 ELF 文件并读取 ELF 文件头、读取节头表、读取节名称字符串表(.shstrtab)。使用.shstrtab 中的字符串比较节名，遍历所有节头，查找.symtab(符号表)和.strtab(字符串表)节，查找成功即可分别读取.strtab 和.symtab 到缓冲区。

以上操作获取足够信息，即可开始查找匹配的符号，只处理函数符号，此时分为两种情况：符号大小为 0（如内联函数、汇编函数），要求 `st_value == return_address`，需要实现精确匹配，即仅当地址完全相等时匹配；符号有确定大小，要求 `st_value <= return_address < st_value + st_size`，即返回地址在函数代码范围内。当多个符号都匹配时，选择 `st_value` 最大的（即起始地址最高的）以确保找到的是最内层、最接近返回地址的函数。具体实现代码如下：

```
Elf64_Sym *best_match = NULL;

for (int i = 0; i < num_symbols; i++) {
    uint8_t type = ELF64_ST_TYPE(symtab_buf[i].st_info);
    if (type != STT_FUNC) continue;
    if (symtab_buf[i].st_value == 0) continue;

    if (symtab_buf[i].st_value <= return_address) {
        if (symtab_buf[i].st_size == 0) {
            if (symtab_buf[i].st_value == return_address) {
                if (!best_match || symtab_buf[i].st_value > best_match->st_value) {
                    best_match = &symtab_buf[i];
                }
            }
        } else {
            if (return_address < symtab_buf[i].st_value + symtab_buf[i].st_size) {
                if (!best_match || symtab_buf[i].st_value > best_match->st_value) {
                    best_match = &symtab_buf[i];
                }
            }
        }
    }
}
```

若找到匹配符号，从 `strtab_buf` 中获取符号名，将符号名安全地复制到 `func_name_buf` 以防止缓冲区溢出，设置终止符，关闭 ELF 文件，返回结果指针即函数名字符串地址；若未成功匹配返回 `NULL` 即可。

2. 实验结果

应用程序调用 `print_backtrace` 时，应能够通过控制输入的参数控制回溯的层数。例如，如果参数为 5 则只输出 5 层回溯（参数<=实际层数）；如果参数为 100（参数>实际层数），则应只回溯到 `main` 函数就停止回溯。

测试调用 `print_backtrace(5)`: 只输出 5 层回溯, 结果与预期相符。

```
gabriel@LAPTOP-CT8H0900:~/riscv-pke$ spike ./obj/riscv-pke ./obj/app_print_backtrace
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
Application: ./obj/app_print_backtrace
Application program entry point (virtual address): 0x00000000810000a2
Switch to user mode...
back trace the user app in the following:
f8
f7
f6
f5
f4
User exit with code:0.
System is shutting down with exit code 0.
```

图 2 测试调用 `print_backtrace(5)`

测试调用 `print_backtrace(100)`: 回溯到 `main` 函数就停止回溯, 结果与预期相符。

```
gabriel@LAPTOP-CT8H0900:~/riscv-pke$ spike ./obj/riscv-pke ./obj/app_print_backtrace
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
Application: ./obj/app_print_backtrace
Application program entry point (virtual address): 0x00000000810000a4
Switch to user mode...
back trace the user app in the following:
f8
f7
f6
f5
f4
f3
f2
f1
main
User exit with code:0.
System is shutting down with exit code 0.
```

图 3 测试调用 `print_backtrace(100)`

本地调试 `git` 记录:

```
gabriel@LAPTOP-CT8H0900:~/riscv-pke$ git log
commit 670fcf9cf1c394fe9b85745690c5d0beea5064bd (HEAD -> lab1_challenge1_backtrace, origin/lab1_challenge1_backtrace)
Author: gabriel <3481963967@qq.com>
Date: Wed Dec 24 15:28:17 2025 +0800

    my work on lab1_challenge1 is done.

commit 6edf9c41160038afc117da0c86e3940d5fc9d4aa
Merge: ff745a9 9d3384c
Author: gabriel <3481963967@qq.com>
Date: Tue Dec 9 15:53:48 2025 +0800

    continue to work on lab1_challenge1
```

图 4 lab1 git log

1.3 实验调试及心得

本次实验使用 AI 辅助完成。AI 类型：ChatGPT

本次实验在由函数返回地址获取函数名的实现中使用 AI 辅助完成，实验中指导 AI 完成上述函数，并根据代码整体框架以及代码风格，在生成代码已经能够完成实验任务的基础上对生成代码进行再次规范完成最终版本的代码。



由于完整问答过长，将完整问答贴于附件 [ELF64 查找函数名.pdf](#) 中，需要查看电子版获取。此处提供部分截图作为证明，并贴出交流的要点思维链。

我：假设你是一个操作系统内核开发人员，现在你需要基于 64 位 RISC-V 架构进行操作系统内核开发。首先你需要了解 RISC-V-64 架构中 ELF 文件的 Section Header 的结构，String Table 的结构。已知通过 String Table 可以读取每个 Section Header 对应的 Section 的名字，通过 C 语言的字符串比较函数，可以读取我们想要的 Section 的内容。对于 .symtab 和 .strtab，查询 .symtab 表中每一个 symbols 的结构定义，结合 .strtab 找到这个符号对应的名字。根据以上信息请用 C 语言完成已知一个地址，找到对应的函数名的函数。

AI：下面我按内核开发者视角，结合 RISC-V 64 位 ELF（ELF64）规范，一步步说明 Section Header / String Table / Symbol Table 的结构，最后给出一个完整可用的 C 函数：已知一个地址（PC），在 ELF 中查找它属于哪个函数，并返回函数名。

我：已知地址转换函数名的实现函数参数为函数返回地址，返回结果为函数名，ELF 文件路径可以通过函数 `static size_t parse_args(arg_buf *arg_buf_msg)` 解析找到，其中输入命令形式如 `spike ./obj/riscv-pke ./obj/app_print_backtrace`，请找到正确 ELF 文件路径并读取相关内容，完善从返回地址获取函数名的函数。

AI：下面我站在 PKE / RISC-V 64 位教学内核开发者的语境下，把这件事完整串起来，从 命令行解析 → ELF 路径确定 → 读取 ELF → 根据返回地址解析函数名，给你一份可以直接放进内核用的实现思路与代码骨架。

我：为了简化对 .h 文件修改，请不要声明除了名字查找函数以外的其他函数，系统框架中已经提供可用的函数（此处贴出 `spike_interface/spike_file.c` 文件内容）请根据上述信息再次修改由函数地址获取函数名字的代码。

AI：好，这个约束说得非常清楚了。我按 “只实现一个名字查找函数，不新增任何其他函数声明” 的要求，在函数体内部完成所有事情，并且只使用你给出的 Spike 文件接口（`spike_file_open / pread / lseek / close` 等），不依赖额外 helper。下面给你的是 “可以直接放进 PKE 内核源码” 的版本。

以下是部分截图，完整交互过程请查看电子版附录：

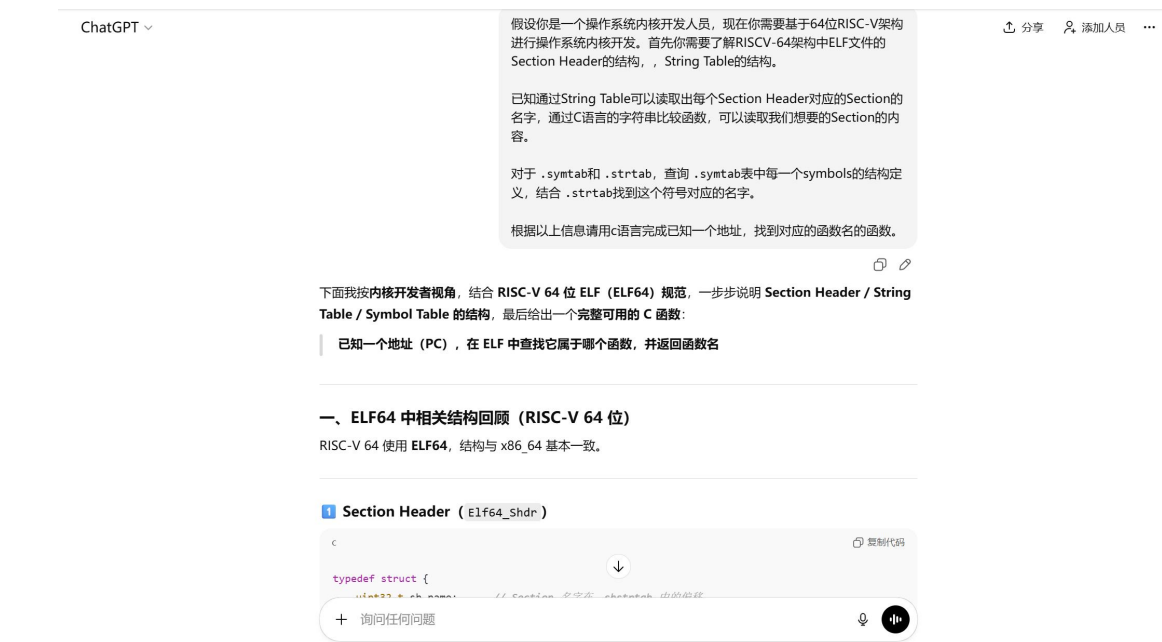


图 5 lab1_challenge1 截图证明

使用 AI 最终给出的完整代码完成实验，多次多条件测试结果均与预期相符。以下以调用函数 `print_backtrace(7)`为例给出测试结果：

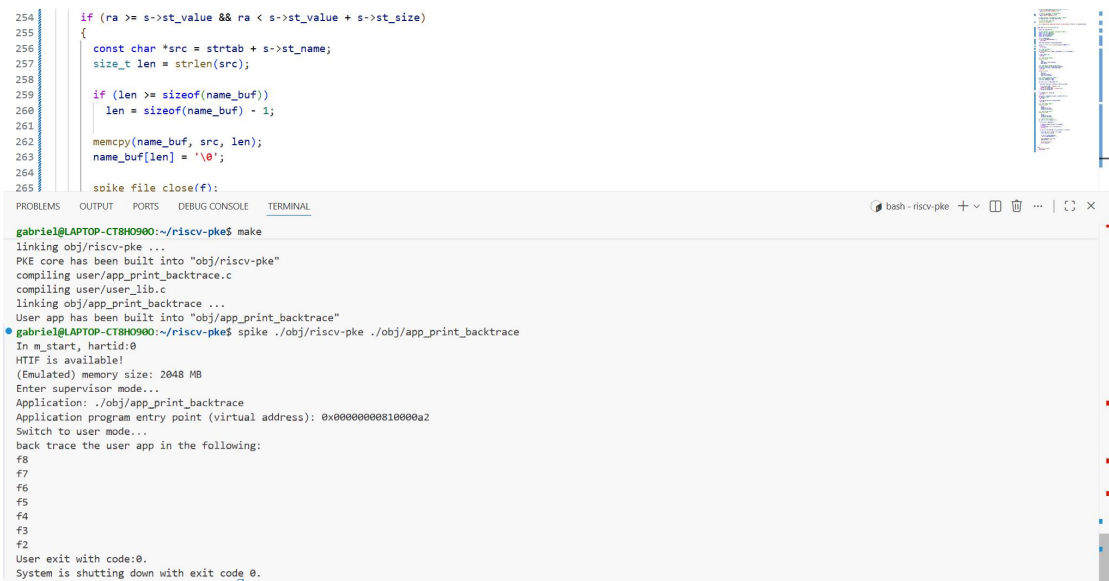


图 6 AI 辅助完成测试

实验心得：本次在基础的完成实验之上使用了 AI 辅助方法，快速完成了由函数返回地址获取函数名功能的实现。在与 AI 的交互过程中，需要给出完整且表述清晰的提示词，不断调试找到 AI 代码中的问题并进行反馈。由于 AI 不了解整个代码框架，在使用 AI 初步生成的代码过程中易产生不兼容问题，因此在使用过程中需要根据运行结果以及报错信息等分析 AI 生成的代码的问题所在，找到对应的框架中代码部分向 AI 反馈以实现最终兼容的正确版本。

实验二 lab2_challenge1 复杂缺页异常

2.1 实验目的

本实验旨在深入理解操作系统中缺页异常的处理机制，特别是针对不同场景下的缺页异常进行区分与处理。通过修改 PKE 内核代码，实现对用户栈空间动态扩展的合理支持，以及对非法地址访问的识别与报错。

本次实验需要理解缺页异常的分类与处理策略，通过分析应用程序中由递归调用导致的栈空间不足，以及数组越界访问引发的非法地址缺页，学习如何在内核中区分合法与非法的缺页请求，并采取不同的处理方式。当用户进程因递归调用导致栈空间不足时，内核应能够捕获缺页异常，分配新的物理页，并将其映射到用户栈的虚拟地址空间，从而支持栈的动态增长。当应用程序访问未映射的或越界的虚拟地址时，内核应能识别此类非法访问，并终止进程的执行，同时给出清晰的错误提示，避免系统进入不可控状态。

通过修改进程数据结构、完善缺页异常处理函数，以及合理管理虚拟地址空间与物理页的映射关系，加深我们对操作系统内存管理整体架构的理解。通过实际编写和测试内核代码，学习如何在复杂场景下定位和解决内存访问异常，为后续更深入的系统开发与调试工作积累经验。本实验在 lab2_3 已实现的缺页异常处理基础上，进一步挑战多场景异常处理的综合能力，为理解现代操作系统的内存保护与动态管理机制提供实践平台。

2.2 实验内容

1. 实验过程

底层分析：堆（Heap）和栈（Stack）是计算机内存管理中两种不同的内存区域，栈存储函数调用信息、局部变量、返回地址等，堆存储动态分配的数据；栈的分配与回收由系统自动管理，大小固定，堆的分配与回收由程序员手动管理，大小可变。因此在访问栈顶附近未映射地址时为合法访问，内核应扩展栈，可以通过分配新物理页并映射实现；访问未分配或已释放的堆地址为非法访问，是程序错误，应终止进程。

由基础实验已经得到对于应用往未被映射的内存空间“写”即缺页异常的处理由函数 `handle_user_page_fault` 完成，因此需要修正该函数内对逻辑地址的检查逻辑，保证其合法性。

对于栈结构，在 `process` 结构体添加 `uint64 size` 项设置每个进程用户栈最大可用页数，并在初始化进程的 `load_user_program` 函数中将其初始化为 20 页但仅作

为虚拟空间预留而不做实际物理页分配。经过以上设置即可对自动分配栈空间进行处理：当逻辑地址超出占空间最高地址，该地址不合法，当逻辑地址位于栈空间合法分配范围内可分配物理页并进行映射。具体处理代码如下：

```
if (stval > USER_STACK_TOP) {
    panic("invalid memory access at address 0x%x\n", stval);
}

if(stval <= USER_STACK_TOP && stval > USER_STACK_TOP - current->size) {
    void* pa = alloc_page();
    uint64 va_page = stval & ~(PGSIZE - 1);
    user_vm_map((pagetable_t)current->pagetable, va_page, PGSIZE, (uint64)pa, prot_to_type(PROT_WRITE |
PROT_READ, 1));
}
```

对于堆结构，通过基础实验分析已经可以得到堆顶由全局变量 `g_ufree_page` 维护，因此低于此地址部分为已经分配的合法堆地址，高于此地址且不在可分配的栈地址之内的地址为不合法的访问地址，需要终止进程。具体实现代码如下：

```
else if(stval <= USER_STACK_TOP - current->size && stval >= g_ufree_page) {
    panic("this address is not available!");
}
```

2. 实验结果

观察应用程序 `app_sum_sequence` 可以得到创建数组时，我们使用了当前的 `naive_malloc` 函数为 `ans` 申请了一个页面（4KB）的大小，对应可以存储的个数上限为（64 位架构 `int` 为 4B，故 $4KB/4B=1024$ ）1024，但在函数调用时，我们试图计算 1025 求和，即越界访问到 `ans[1024]`，因此应出现非法访问的报错信息 `this address is not available!` 并终止进程退出程序。

测试 1025 求和，应报错并终止进程。测试结果与预期相符，结果如下：

```
gabriel@LAPTOP-CT8H0900:~/riscv-pke$ spike ./obj/riscv-pke ./obj/app_sum_sequence
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x0000000080008000, PKE kernel size: 0x0000000000080000 .
free physical memory address: [0x0000000080008000, 0x0000000087ffffff]
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080005000
kernel page table is on
User application is loading.
user frame 0x0000000087fbc000, user stack 0x000000007ffff000, user kstack 0x0000000087fbb000
Application: ./obj/app_sum_sequence
Application program entry point (virtual address): 0x00000000000100a2
Switch to user mode...
handle_page_fault: 000000007ffffdff8
handle_page_fault: 000000007ffffcff8
handle_page_fault: 000000007ffffbfff8
handle_page_fault: 000000007ffffaff8
handle_page_fault: 000000007ffff9fff8
handle_page_fault: 000000007ffff8fff8
handle_page_fault: 000000007ffff7fff8
handle_page_fault: 000000007ffff6fff8
handle_page_fault: 000000000401000
this address is not available!
System is shutting down with exit code -1.
```

图 7 越界访问测试

测试 1024 求和，应不存在错误情况而正常退出程序，计算出求和结果 524800。

测试结果与预期相符，结果如下：

```
gabriel@LAPTOP-CT8H0900:~/riscv-pke$ spike ./obj/riscv-pke ./obj/app_sum_sequence
In m_start, hartid:0
HTIF is available!
(Emulated) memory size: 2048 MB
Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x0000000080008000, PKE kernel size: 0x0000000000008000 .
free physical memory address: [0x0000000080008000, 0x0000000087ffffff]
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080005000
kernel page table is on
User application is loading.
user frame 0x0000000087fbc000, user stack 0x000000007ffff000, user kstack 0x0000000087fbb000
Application: ./obj/app_sum_sequence
Application program entry point (virtual address): 0x0000000000100a2
Switch to user mode...
handle_page_fault: 000000007fffdff8
handle_page_fault: 000000007fffcff8
handle_page_fault: 000000007fffbff8
handle_page_fault: 000000007fffafe8
handle_page_fault: 000000007fff9ff8
handle_page_fault: 000000007fff8ff8
handle_page_fault: 000000007fff7ff8
handle_page_fault: 000000007fff6ff8
Summation of an arithmetic sequence from 0 to 1024 is: 524800
User exit with code:0.
System is shutting down with exit code 0.
```

图 8 边界条件测试

本地调试 git 记录：

```
gabriel@LAPTOP-CT8H0900:~/riscv-pke$ git log
commit 0bd6660afdf6f07b4fef5e8d4e1de899de1e742f (HEAD -> lab2_challenge1_pagefaults, origin/lab2_challenge1_pagefaults)
Author: gabriel <3481963967@qq.com>
Date: Sun Dec 28 18:12:27 2025 +0800

    fix lab2_c1.

commit 4549cc101665f41f1751a18d5d8c7bd60fc1dbe6
Author: gabriel <3481963967@qq.com>
Date: Wed Dec 24 23:19:41 2025 +0800

    my work on lab2_challenge1 is done.

commit b199de661a4fb8f40a5e50a7b28557d9eebdba84
Merge: be365de 5cae99a
Author: gabriel <3481963967@qq.com>
Date: Wed Dec 24 20:37:12 2025 +0800

    continue to work on lab2_challenge1
```

图 9 lab2 git log

2.3 实验调试及心得

本次实验在最初对栈与堆的分布理解不清晰，导致最初仅对地址进行了是否超过栈最高地址以及是否位于未分配的堆顶 `g_ufree_page` 处导致越界访问，此判断条件虽然可以通过测试，但存在严重逻辑漏洞。在后续反思过程中，深入理解栈与堆的结构以及越界访问产生的原因，修复逻辑漏洞，设计并通过全面的测试。

通过本次实验，我深入理解了虚拟内存的工作原理，掌握了栈和堆的动态管理技术，学会了如何设计鲁棒的错误处理机制，培养了从系统角度思考问题的能力，认识到一个好的操作系统不仅要功能正确，还要能够优雅地处理各种异常情况，保护系统安全，同时为用户提供有用的反馈。

实验三 lab3_challenge1 进程等待和数据段复制

3.1 实验目的

本实验旨在深入理解操作系统中的进程同步机制与内存隔离原理，通过实现进程等待 `wait` 系统调用及完善 `fork` 操作中的数据段复制功能，掌握多进程环境下父子进程间的协同与资源管理方法。

通过实现 `wait` 系统调用，学习父进程如何等待子进程结束、回收子进程资源，并正确处理僵尸进程。掌握进程内存空间的隔离与复制，深入理解进程间内存空间的隔离机制。通过在内核中添加 `wait` 系统调用并完善 `do_fork` 函数，学习如何在内核层面支持进程管理和内存操作，提升操作系统内核的开发与调试能力。结合实验前期对 `ELF` 文件结构和虚拟内存管理的理解，进一步实现数据段的正确加载与映射，巩固从应用层到内核层的完整执行流程认知。

通过本实验，我们能够实现进程间的同步等待与资源回收机制；理解并实现 `fork` 过程中数据段的独立复制，保证进程间数据的隔离性；进一步掌握操作系统内核的开发方法，为后续更复杂的进程管理与内存机制实验奠定基础。

3.2 实验内容

1. 实验过程

Part1.补充 `do_fork` 函数

补充 `do_fork` 函数，由于基础实验仅实现了代码段的复制，我们需要继续实现数据段的复制并保证 `fork` 后父子进程的数据段相互独立。对于各个段的处理方式与处理原因如下表：

表 1 子进程段处理设计

段类型	处理策略	权限	原因
CONTEXT	直接复制	读写	继承执行上下文
STACK	复制内容	读写	独立运行时栈
HEAP	选择性复制	读写	复制活跃堆，跳过空闲块
CODE	共享映射	只读+执行	代码不变，节省内存
DATA	完整复制	读写	数据隔离，独立修改

因此子进程继承父进程的虚拟地址空间布局，并且对数据段每个页进行完整复制，保证数据隔离性，具体实现代码如下：

```
case DATA_SEGMENT:
    uint64 data_va = parent->mapped_info[i].va;
    for (int pg = 0; pg < parent->mapped_info[i].npages; pg++) {
        uint64 page_va = data_va + pg * PGSIZE;
        uint64 page_pa = lookup_pa(parent->pagetable, page_va);
        void* new_pa = alloc_page();
        memcpy(new_pa, (void*)page_pa, PGSIZE);
        user_vm_map((pagetable_t)child->pagetable, page_va, PGSIZE,
                    (uint64)new_pa, prot_to_type(PROT_WRITE | PROT_READ, 1));
    }

    child->mapped_info[child->total_mapped_region].va = parent->mapped_info[i].va;
    child->mapped_info[child->total_mapped_region].npages =
        parent->mapped_info[i].npages;
    child->mapped_info[child->total_mapped_region].seg_type = DATA_SEGMENT;
    child->total_mapped_region++;
    break;
```

Part2.wait 函数实现

由于函数在系统调用路径上的完善过程已经在实验一 lab1_challenge1 打印用户程序调用栈中分析过，在此不再赘述，最终的实现是在 syscall.c 中完成函数 `ssize_t sys_user_wait(int pid)`。

当 `pid` 为 -1 时，父进程等待任意一个子进程退出即返回子进程的 `pid`。只需要遍历 `procs[NPROC]` 找到一个 `parent` 为当前进程（即检查 `pid` 对应的进程是否为当前进程的子进程）且处于 `ZOMBIE` 状态的进程，释放该进程资源，返回该子进程 `pid`，结束等待状态即可。若未找到上述情况进程，则循环等待，阻塞当前进程并将当前进程状态设置为 `BLOCKED`，调用 `schedule` 函数进行进程调度切换到其他就绪进程。

当 `pid` 不为 -1 时，检查 `pid` 是否合法，检查 `pid` 对应的进程是否为当前进程的子进程，代码如下：

```
if(pid < 0 || pid >= NPROC || procs[pid].parent != current){
    return -1;
}
```

当 `pid` 大于 0 时，父进程等待进程号为 `pid` 的子进程退出即返回子进程的 `pid`。首先进行判定是否需要进行循环等待，即只要指定子进程不是僵尸状态就继续等待，阻塞当前进程并将当前进程状态设置为 `BLOCKED`，调用 `schedule` 函数进行进程调度切换到其他就绪进程。当子进程为僵尸进程即释放子进程资源，返回子进程 `pid`，结束等待状态即可。

由于在相应子进程退出时需要结束父进程的等待状态，需要补全进程退出函数 `sys_user_exit`。当等待的子进程退出时，需要检查其父进程是否处于被阻塞的 `BLOCKED` 状态，若是则需要修改父进程状态为 `READY` 并插入就绪队列即可。具体实现代码如下：

```
if (current->parent && current->parent->status == BLOCKED) {  
    current->parent->status = READY;  
    insert_to_ready_queue(current->parent);  
}
```

2. 实验结果

分析 `app_wait.c` 文件，父进程的 `flag` 为 0，首先创建子进程，之后父进程调用函数 `wait(-1)` 进入等待任意子进程结束的状态，等待结束后执行 `printu`，之后调用 `exit(0)` 退出进程结束程序；创建的子进程中 `flag` 为 1，首先创建孙子进程，之后子进程调用函数 `wait(pid)` 进入等待孙子进程结束的状态，等待结束后执行 `printu`，之后调用 `exit(0)` 退出进程结束子进程；创建的孙子进程中 `flag` 为 2，执行 `printu` 结束后调用 `exit(0)` 退出进程结束孙子进程。由于进程之间数据独立且隔离，因此可以得到正确的输出应该为：Grandchild2→Child1→Parent0。

执行代码，可以看到子进程创建正确，进入就绪队列顺序与时机正确，输出数据正确，实验结果与预期相符。实验结果如下：

```
gabriel@LAPTOP-CT8H0900:~/riscv-pke$ spike ./obj/riscv-pke ./obj/app_wait  
In m_start, hartid:0  
HTIF is available!  
(Emulated) memory size: 2048 MB  
Enter supervisor mode...  
PKE kernel start 0x0000000080000000, PKE kernel end: 0x000000008000b000, PKE kernel size: 0x000000000000b000  
free physical memory address: [0x000000008000b000, 0x0000000087ffffff]  
kernel memory manager is initializing ...  
KERN_BASE 0x0000000080000000  
physical address of _etext is: 0x0000000080005000  
kernel page table is on  
Switch to user mode...  
in alloc_proc. user frame 0x0000000087fbc000, user stack 0x000000007ffff000, user kstack 0x0000000087fbb000  
User application is loading.  
Application: ./obj/app_wait  
CODE_SEGMENT added at mapped info offset:4  
DATA_SEGMENT added at mapped info offset:5  
Application program entry point (virtual address): 0x0000000000100b0  
going to insert process 0 to ready queue.  
going to schedule process 0 to run.  
User call fork.  
will fork a child from parent 0.  
in alloc_proc. user frame 0x0000000087fae000, user stack 0x000000007ffff000, user kstack 0x0000000087fad000  
going to insert process 1 to ready queue.  
going to schedule process 1 to run.  
User call fork.  
will fork a child from parent 1.  
in alloc_proc. user frame 0x0000000087fa1000, user stack 0x000000007ffff000, user kstack 0x0000000087fa0000  
going to insert process 2 to ready queue.  
going to schedule process 2 to run.  
Grandchild process end, flag = 2.  
User exit with code:0.  
going to insert process 1 to ready queue.  
going to schedule process 1 to run.  
Child process end, flag = 1.  
User exit with code:0.  
going to insert process 0 to ready queue.  
going to schedule process 0 to run.  
Parent process end, flag = 0.  
User exit with code:0.  
no more ready processes, system shutdown now.  
System is shutting down with exit code 0.
```

图 10 wait 函数测试

本地调试 git 记录:

```
gabriel@LAPTOP-CT8H0900:~/riscv-pke$ git log
commit 74f936cad7e0d4286be94c5e0c9f8cc4c6ca9885 (HEAD -> lab3_challenge1_wait, origin/lab3_challenge1_wait)
Author: gabriel <3481963967@qq.com>
Date:   Fri Dec 26 13:49:05 2025 +0800

    my work on lab3_challenge1 is done.

commit 819398a2491e725692124d763fb234a6847e91ee
Merge: 18ef8d7 788ffba
Author: gabriel <3481963967@qq.com>
Date:   Thu Dec 25 19:34:58 2025 +0800

    continue to work on lab3_challenge1
```

图 11 lab3 git log

3.3 实验调试及心得

1. 试验调试

在最初完善数据段复制与实现 wait 函数后，调用顺序依然存在问题。部分错误输出如下：

```
Application program entry point (virtual address): 0x00000000000100b0
going to insert process 0 to ready queue.
going to schedule process 0 to run.
User call fork.
will fork a child from parent 0.
in alloc_proc. user frame 0x0000000087fae000, user stack 0x000000007ffff000, user kstack 0x0000000087fad000
going to insert process 1 to ready queue.
going to schedule process 1 to run.
User call fork.
will fork a child from parent 1.
in alloc_proc. user frame 0x0000000087fa1000, user stack 0x000000007ffff000, user kstack 0x0000000087fa0000
going to insert process 2 to ready queue.
going to schedule process 2 to run.
Grandchild process end, flag = 2.
User exit with code:0.
ready queue empty, but process 0 is not in free/zombie state:3
ready queue empty, but process 1 is not in free/zombie state:3
Not handled: we should let system wait for unfinished processes.
```

图 12 lab3 错误输出

分析输出结果可以看到被挂起的父进程与子进程未被再次唤醒，考虑到唤醒时机应该在对应的子进程退出时，因此需要修改 exit 函数的实现，保证在退出后能够正确将对应的挂起的父进程唤醒。经过上述修改后输出正确。

2. 实验心得

通过本次实验，我深刻体会到 wait 的实现保障了系统的安全稳定。exit 执行时，进程的内核栈正用于执行系统调用，立即销毁可能导致严重后果。将其标记为 ZOMBIE，把真正释放延迟到父进程的 wait 中执行是一个更安全的处理。循环等待检查中在每次被调度器唤醒后，都必须重新验证，因为进程状态可能在睡眠期间被其他事件改变，这让我意识到了内核编程中考虑竞态条件的必要性。本实验通过实现 wait 系统调用和完善 fork 的数据段处理，构建了一个完整的进程同步和资源管理框架，为理解现代操作系统的内部机制提供了实践基础。

实验四 lab4_challenge1 相对路径

4.1 实验目的

本实验旨在深入理解文件系统中路径解析机制的实现原理，特别是在进程上下文中支持相对路径访问的关键技术。通过实现相对路径的解析与处理，我们可以掌握操作系统如何维护进程的当前工作目录（Current Working Directory, CWD），并在此基础上实现对文件与目录的灵活访问。

实验中，我们可以通过扩展 PKE 文件系统的路径处理逻辑，使系统能够正确解析以 ./、../ 开头的相对路径字符串，并将其转换为对应的绝对路径。本次实验需要深入理解进程文件管理结构中 CWD 的作用与维护方式；完成虚拟文件系统（VFS）中相关函数的扩展，使其能够基于 CWD 进行路径解析；完成系统调用机制的扩展，新增用于读取和修改 CWD 的系统调用 rcwd、ccwd，并在用户层实现对应的库函数 pwd、cd。

完成本实验，我们可以加深对文件系统层次化设计的理解，进一步掌握操作系统如何在多进程环境中为每个进程维护独立的文件系统状态（如 CWD），从而实现更符合实际应用场景的文件访问方式。同时，这也为后续实现更复杂的 Shell 与命令执行环境奠定重要基础。

4.2 实验内容

1. 实验过程

Part1.提供解析相对路径的支持

由于需要支持 ./ ../ ... 的解析，需要在 vfs 层提供的接口中进行修改以支持相对路径解析。vfs 提供的接口在 vfs.c 文件中，其中需要进行路径解析的函数如下表：

表 2 需要路径解析的接口

file *vfs_open(const char *path, int flags)
int vfs_link(const char *oldpath, const char *newpath)
int vfs_unlink(const char *path)
struct file *vfs_opendir(const char *path)
int vfs_mkdir(const char *path)
struct dentry *lookup_final_dentry(const char *path, struct dentry **parent, char *miss_name)

需要对上述包含路径解析步骤的函数进行修改。其中 lookup_final_dentry 函数被其他函数调用传入父节点，因此与其他函数修改逻辑有所不同。修改代码以支持相对路径解析的逻辑为当识别路径为相对于当前路径时，parent 保持不变，

将 path 后移即可；当识别路径为相对于上一级路径时，parent 修改为父节点，将 path 后移。具体实现代码如下：

```
if(path[0] == '.' && path[1] == '/') {
    *parent = *parent;
    path = path + 2;
}
if(path[0] == '.' && path[1] == '.' && path[2] == '/') {
    *parent = (*parent)->parent;
    path = path + 3;
}
if(path[0] == '.' && path[1] == '\\0') {
    *parent = *parent;
    return *parent;
}
if(path[0] == '.' && path[1] == '.' && path[2] == '\\0') {
    *parent = (*parent)->parent;
    return *parent;
}
```

其他函数由于系统默认传入父节点为根节点，当时别到路径为相对于当前的路径时需要将 parent 更新为当前节点即 `parent = current->pfiles->cwd`；当时别到路径为相对于上一级的路径是需要将 parent 更新为当前节点的父节点即 `parent = current->pfiles->cwd->parent`，其余处理逻辑与 `lookup_final_dentry` 函数一致，此处不再赘述。

Part2.完成用户层 pwd 函数（显示进程当前工作目录）

由于函数在系统调用路径上的完善过程已经在实验一 lab1_challenge1 打印用户程序调用栈中分析过，在此不再赘述，最终的实现是在 `syscall.c` 中完成函数 `ssize_t sys_user_read_cwd(char *pathva)`，由于需要获取当前路径，为保证良好的封装与结构，系统函数需要调用 `proc_file.c` 文件中定义的操作，因此将保存路径的字符串虚拟地址转为物理地址并获取当前路径后将调用 `do_rcwd` 函数进行进一步解析。而 `proc_file.c` 中的动作又会调用 `vfs` 中定义的数据结构，将动作进一步传导到具体的文件系统，因此 `do_rcwd` 函数需要调用 `vfs.c` 文件中的 `vfs_rcwd` 函数进行最终的获取路径功能的实现。

vfs_rcwd 函数的实现逻辑：当传入参数异常即当前路径不存在或解析结果存放的字符串地址不存在时出错，退出程序。当当前路径为跟目录时，装入字符串“/”即可。对于一般情况，从当前节点逐个遍历其父节点，将路径名称逐个拼接，直到回溯至根节点完成遍历即可，该部分具体代码如下：

```
struct dentry *cur = cwd;
char temp_buf[MAX_PATH_LEN];
temp_buf[0] = '\\0';
```

```

while (cur != vfs_root_dentry) {
    char new_buf[MAX_PATH_LEN];
    strcpy(new_buf, "/");
    strcat(new_buf, cur->name);
    strcat(new_buf, temp_buf);
    strcpy(temp_buf, new_buf);
    cur = cur->parent;
}
strcpy(pathbuf, temp_buf);

```

Part3.完成用户层 cd 函数（切换进程当前工作目录）

由于函数在系统调用路径上的完善过程已经在实验一 lab1_challenge1 打印用户程序调用栈中分析过，系统函数到 vfs 层函数的调用逻辑已经在上述对 pwd 函数实现中分析过，此处不再赘述，最终的实现是在 vfs.c 文件中完成切换目录函数 `int vfs_ccwd(struct dentry *cwd, char *pathbuf)`。

`vfs_ccwd` 函数的实现逻辑：当传入参数异常即当前路径不存在或目标路径存放的字符串地址不存在时出错，退出程序。当目标路径第一个字符为/时，表示路径从根目录开始，因此设置 `parent` 为根目录；其他情况默认从当前目录开始，因此设置 `parent` 为 `cwd`。调用 `lookup_final_dentry` 传入目标路径字符串以及 `parent` 参数即可找到目标路径节点，将当前节点设置为上述目标节点即可，具体实现核心代码如下：

```

struct dentry *parent;
char miss_name[MAX_PATH_LEN];

if (pathbuf[0] == '/') {
    parent = vfs_root_dentry;
} else {
    parent = cwd;
}

struct dentry *target_dentry = lookup_final_dentry(pathbuf, &parent, miss_name);

if (!target_dentry) {
    sprintf("vfs_ccwd: cannot find the directory: %s\n", pathbuf);
    return -1;
}

if (target_dentry->dentry_inode->type != DIR_I) {
    sprintf("vfs_ccwd: %s is not a directory!\n", pathbuf);
    return -1;
}

current->pfiles->cwd = target_dentry;
return 0;

```

2. 实验结果

运行测试，各项功能均正确实现，实验结果与预期相符。实验结果如下：

```
gabriel@LAPTOP-CT8H0900:~/riscv-pke$ spike ./obj/riscv-pke ./obj/app_relativepath
Enter supervisor mode...
PKE kernel start 0x0000000080000000, PKE kernel end: 0x0000000080011000, PKE kernel size: 0x0000000000011000 .
free physical memory address: [0x0000000080011000, 0x0000000087ffffff]
kernel memory manager is initializing ...
KERN_BASE 0x0000000080000000
physical address of _etext is: 0x0000000080009000
kernel page table is on
RAMDISK0: base address of RAMDISK0 is: 0x0000000087f35000
RFS: format RAMDISK0 done!
Switch to user mode...
in alloc_proc. user frame 0x0000000087f29000, user stack 0x000000007ffff000, user kstack 0x0000000087f28000
FS: created a file management struct for a process.
in alloc_proc. build proc_file_management successfully.
User application is loading.
Application: ./obj/app_relativepath
CODE_SEGMENT added at mapped info offset:4
DATA_SEGMENT added at mapped info offset:5
Application program entry point (virtual address): 0x00000000000100ec
going to insert process 0 to ready queue.
going to schedule process 0 to run.

===== Test 1: change current directory =====
cwd:/
change current directory to ./RAMDISK0
cwd:/RAMDISK0

===== Test 2: write/read file by relative path =====
write: ./ramfile
file descriptor fd: 0
write content:
hello world
read: ./ramfile
read content:
hello world

===== Test 3: Go to parent directory =====
cwd:/RAMDISK0
change current directory to ..
cwd:/
read: ./hostfile.txt
file descriptor fd: 0
read content:
This is an apple.
Apples are good for our health.

All tests passed!

User exit with code:0.
no more ready processes, system shutdown now.
System is shutting down with exit code 0.
```

图 13 路径解析测试

本地调试 git 记录：

```
gabriel@LAPTOP-CT8H0900:~/riscv-pke$ git log
commit ad63fdad14542d5a0ea821bd16900c970685d6b2 (HEAD -> lab4_challenge1_relativepath, origin/main, origin/lab4_challenge1_relativepath)
Author: gabriel <3481963967@qq.com>
Date: Sat Dec 27 22:57:58 2025 +0800

    my work on lab4_challenge1 is done.

commit d354c5c63502d4268cde4c53e0b38367db452716
Merge: b047d4e afee75f
Author: gabriel <3481963967@qq.com>
Date: Sat Dec 27 02:33:43 2025 +0800

    continue to work on lab4_challenge1
```

图 14 lab4 git log

4.3 实验调试及心得

1. 实验心得

完成本次相对路径实验后，我对操作系统中文件系统的路径管理机制有了更为深入和系统的理解。在基础的文件操作实验中，我们已熟悉了从根目录开始的绝对路径访问，而本次实验最大的认知突破在于理解了当前工作目录（CWD）作为进程执行上下文一部分的重要性。我认识到，文件系统不仅要管理静态的磁盘数据，更要为每个动态运行的进程进行维护。通过将 CWD 由固定的根目录改为可动态指向任一 `dentry` 的指针，我真正体会到操作系统如何通过精巧的数据结构，在统一的全局文件树之上，为每个进程营造出个性化的、基于相对路径的工作空间，这让我深刻理解了进程隔离性在文件访问层面的具体体现。

在实现路径解析时，首先需要判断路径是绝对路径（以 `/` 开头）还是相对路径。对于相对路径，其解析的起点不再是固定的根目录 `vfs_root_dentry`，而是进程自己的 `current->pfiles->cwd`。这一逻辑的修改让我意识到，操作系统接口的灵活性源于底层对起点的灵活定义。沿着 `dentry->parent` 指针上溯的过程让我体会到目录树在内存中的链接结构如何被实际运用。

本次实验让我深刻体会到通过严谨的数据结构和算法，将复杂逻辑封装为简洁、可靠的用户接口的设计理念。

2. 实验建议

本次实验内容丰富，让我更加深刻的学习了操作系统的实现与设计理念。但美中不足的是本次实验仅实现最基本的 `cd`、`pwd` 功能，而在 Linux 操作系统的使用中，创建文件夹 `mkdir`、列出当前目录下的文件 `ls` 等操作极为常用，可以添加上述功能的测试更进一步全面的验证相对路径解析识别的正确性。在使用 Linux 操作系统的过程中，命令常常带有参数，参数的设计增强了命令的丰富性，在实验中可以添加对命令参数的支持，实现在更深入了解操作系统的同时更加贴近真实的操作系统。